Software Test Plan (STP)

Multimodal Behavioral Cloning for a Robotic Arm Using Mamba and OpenCLIP

Oria Cohen | Computer Science Final Project

Project	RoboMamba: Multimodal Behavioral Cloning for a Robotic Arm Using Mamba and OpenCLIP
Student	Oria Cohen
Document Type	Software Test Plan - styled final documentation version
System Type	Offline multimodal robotic imitation learning pipeline
Primary Test Focus	Dataset, YOLO filtering, OpenCLIP embeddings, Mamba training/evaluation, Streamlit UI, results visualization

1. Introduction

This document presents the Software Test Plan (STP) for the RoboMamba project. The purpose of this test plan is to define how the system will be tested, which components will be verified, and what expected behavior should be confirmed during testing.

The software to be tested is a multimodal robotic imitation learning pipeline that combines image sequences and robotic state data in order to predict the next robotic action. The system includes dataset selection, YOLO-based data filtering, OpenCLIP embedding generation, Mamba model training or checkpoint selection, model evaluation, and results visualization through a Streamlit user interface.

2. Test Items

Test Item	Description
Streamlit User Interface Module	Operates the pipeline, navigates between stages, configures parameters, displays progress, logs, metrics, charts, prediction examples, and final results.
Dataset Selection and Validation Module	Selects the dataset folder, detects valid trajectory folders and existing CSV files, and validates whether the selected structure can be used.
YOLO-Based Data Filtering Module	Scans robotic trajectories, filters valid samples, detects successful trajectories, generates or reuses a filtered CSV, and supports safe process behavior.
OpenCLIP Embedding Generation Module	Extracts visual embeddings from image frames, saves cache files, detects existing or partial caches, and supports reuse or recomputation.
Data Loading and Sequence Construction Module	Loads the filtered dataset, synchronizes visual embeddings with robot state/action data, creates fixed-length temporal windows, and prepares samples.
Mamba Model Module	Receives multimodal sequence inputs and predicts the next robotic action using the Mamba sequence model.
Training Module	Trains the Mamba model, applies hyperparameters, saves checkpoints and logs, applies early stopping, and preserves relevant training state.
Checkpoint and Weights Management Module	Detects trained models, displays checkpoint information, loads selected checkpoints, and prevents evaluation when no valid checkpoint is selected.
Evaluation Module	Loads a trained checkpoint, runs evaluation, computes metrics, saves prediction examples, and generates evaluation logs.
Results Visualization Module	Displays metrics, prediction-vs-ground-truth charts, trajectory images, action comparison tables, and evaluation logs.
Process and State Management Module	Manages long-running processes, preserves stage status, detects running or stopped processes, and restores state after reopening the UI.
Logging and Error Handling Module	Presents clear log messages, validation errors, process updates, and user-facing feedback throughout the pipeline.

3. Features to be Tested

Feature	Expected Capability
Dataset Selection and Validation	Select dataset folder, display selected path, detect valid trajectories, and prevent progress when data is invalid or missing.

Feature	Expected Capability
Existing CSV Detection and Reuse	Detect, validate, display, and reuse an existing filtered CSV when valid.
YOLO-Based Data Filtering	Run YOLO-based trajectory validation, generate filtered CSV, update progress and statistics, and classify filtering status.
YOLO Stop, Resume, and Restart	Support safe stopping, partial-result continuation, and restart from the beginning when required.
OpenCLIP Embedding Generation	Generate visual embeddings, save cache files, display progress, and report completion.
OpenCLIP Cache Reuse and Recompute	Detect complete/partial embedding caches, allow reuse, recomputation, and continuation based on existing artifacts.
Data Loading and Sequence Preparation	Load filtered data, synchronize embeddings with robot data, create temporal windows, and prepare samples.
Mamba Model Training	Train with selected hyperparameters, display training progress, save logs/checkpoints, and apply early stopping.
Checkpoint Selection and Validation	Detect checkpoints, display metadata, allow model selection, and prevent evaluation when the checkpoint is missing.
Model Evaluation	Evaluate selected checkpoints, compute MAE and related metrics, save prediction examples, and generate logs.
Results Visualization	Display metrics, charts, prediction examples, trajectory images, action tables, and logs in a clear layout.
UI Navigation and Flow Control	Allow navigation while enforcing stage order and preventing invalid transitions.
Process State Preservation	Preserve completed, running, stopped, partial, and ready states according to real files and processes.
Logging and Error Handling	Display clear logs and errors for invalid input, missing files, failed processes, or incomplete artifacts.

4. Features Not to be Tested

The following features and conditions are outside the scope of this STP or are not part of the final system test focus.

Out-of-Scope Item	Clarification
Real Robot Deployment and Physical Execution	The STP focuses on the offline dataset-based pipeline and does not test deployment on a physical robotic arm.
Real-Time Closed-Loop Robot Control	The system is evaluated using recorded trajectories and ground-truth data rather than live robot commands.
Standalone YOLO Detector Benchmarking	The STP verifies the integrated YOLO filtering stage and use of trained weights, but not YOLO as an independent detection research benchmark.
Fine-Tuning OpenCLIP	OpenCLIP is used as a pretrained frozen visual encoder; OpenCLIP fine-tuning is outside this testing scope.
Comparison with Alternative Sequence Models	Testing focuses on the implemented Mamba-based pipeline rather than full experimental benchmarks against RNN, LSTM, or Transformer models.
Full BridgeData Benchmark Evaluation	Testing focuses on the selected dataset structure and task subset used in the project rather than every BridgeData task and version.
Formal Hardware Stress Benchmarking	The tests validate the project pipeline on the selected large dataset and environment, but do not measure maximum hardware limits or long-duration endurance benchmarks.
Third-Party Library Internal Logic	The internal implementations of PyTorch, OpenCLIP, Streamlit, Plotly, Ultralytics, and Mamba-SSM are not tested; only their integration is verified.
Security and Access Control Testing	Authentication, authorization, penetration testing, and user permission management are outside scope because the system is a local research tool.
Cross-Platform and Browser Compatibility Testing	Testing is performed in the main development environment and browser, not across all operating systems, browsers, screen sizes, or mobile devices.

5. Testing Strategy

The testing strategy combines module-level testing, integration testing, end-to-end system testing, regression testing, and acceptance testing. Because the system is implemented as a modular pipeline, each stage is validated separately and then the complete workflow is tested as one continuous process.

- Unit and module tests validate individual functions such as dataset path validation, CSV structure validation, embedding cache detection, checkpoint metadata loading, sequence window construction, metric calculation, and error handling.
- Integration tests verify consecutive pipeline stages: YOLO output must generate a valid CSV, OpenCLIP must generate embeddings that the DataLoader can consume, and evaluation must load a selected checkpoint and produce UI results.
- End-to-end system tests validate the full Streamlit workflow from dataset selection through YOLO filtering, OpenCLIP embeddings, model training or checkpoint selection, evaluation, and final results visualization.
- Regression tests verify that UI or code changes do not break previously working functionality such as navigation, progress indicators, artifact reuse, checkpoint loading, charts, prediction examples, and Results layout.
- Acceptance tests represent the expected final use of the system: processing valid data, reusing/generating artifacts, running evaluation, displaying metrics and examples, and handling common errors without misleading the user.

6. Test Environment

Area	Environment Details
Hardware	Development machine or VM with NVIDIA GPU/CUDA support, sufficient RAM, and storage for trajectory data, CSV files, embeddings, checkpoints, logs, and outputs.
Operating System	Linux-based development environment with a local desktop or browser client for Streamlit.
Runtime	Python 3.10 or later, Conda or virtual environment, and pip-managed dependencies when required.
Core Libraries	PyTorch, CUDA Toolkit, OpenCLIP, Mamba-SSM, Ultralytics/YOLO, Streamlit, Plotly, NumPy, Pandas, Pillow and/or OpenCV.
Project Data and Artifacts	BridgeData-style trajectory folders, filtered CSV, OpenCLIP embedding cache files, Mamba checkpoints/configurations, training logs, evaluation logs, and prediction examples.
Testing Tools	Streamlit browser interface, command-line scripts for backend validation, file system inspection, and a modern browser such as Google Chrome.

7. Responsibilities

Since this is an individual final project, all testing activities are planned, executed, documented, and reviewed by the project developer, Oria Cohen, as part of the system implementation and validation process.

8. Schedule and Testing Milestones

Milestone	Description
Unit and Module Validation	Validate modules such as dataset validation, CSV detection, embedding cache detection, checkpoint loading, and metric calculation.
Pipeline Integration Testing	Verify integration between Dataset, YOLO filtering, OpenCLIP embedding generation, model training, checkpoint selection, evaluation, and results visualization.
End-to-End System Testing	Run the complete workflow through the Streamlit UI and verify behavior from dataset selection to final results display.
Error Handling and State Recovery Testing	Test invalid inputs, missing files, partial artifacts, stop/resume behavior, and restoration after reopening the UI.
Regression and Final Acceptance Testing	Re-run main scenarios after changes and confirm that the final version is stable and ready for submission.

9. Risks and Contingencies

Risk	Potential Impact	Contingency / Planned Response
Invalid or incomplete dataset structure	Pipeline stages may fail during loading, filtering, embedding generation, or evaluation.	Validate the selected folder before allowing progress and display clear missing-file or structure errors.
Missing or corrupted filtered CSV file	OpenCLIP, training, or evaluation may fail or use invalid data.	Validate the CSV structure before reuse and require rebuilding when the CSV is invalid.
Missing or partial OpenCLIP embedding cache	Training or evaluation may fail because embeddings are unavailable.	Detect complete and partial caches; allow reuse, continuation, or recomputation when needed.
Long runtime for filtering, embeddings, or training	Testing may take longer than expected or be interrupted.	Support safe process handling and use valid existing artifacts when appropriate.
GPU, CUDA, or dependency issues	Model training, embedding extraction, or Mamba execution may fail or run slowly.	Verify CUDA, PyTorch, and package availability and document the environment for reproducibility.

Risk	Potential Impact	Contingency / Planned Response
Missing or outdated checkpoints	Evaluation may use a model that does not match the selected run.	Display checkpoint metadata and verify that the selected weights file exists before evaluation.
Incorrect restoration of process state	The UI may show a misleading completed, partial, running, or ready state.	Restore state using project artifacts, logs, and process status rather than only UI session variables.
UI layout or visualization issues	Results may be difficult to interpret if charts, tables, logs, or images are misaligned.	Run regression checks on the Results screen, charts, prediction examples, tables, and logs.
Low model performance or unstable metrics	The trained model may not reach expected prediction quality.	Report quantitative metrics and inspect saved prediction examples for qualitative validation.
Code changes breaking existing behavior	A fix in one stage may affect another stage.	Perform regression testing after changes, especially for the main end-to-end workflow.

10. Detailed Test Cases

The following test cases define the main validation scenarios for the system. They are organized by pipeline stage and include functional, integration, UI, recovery, and data consistency tests.

10.1 Dataset Selection and Validation Tests

Test ID	Feature	Preconditions	Test Steps	Expected Result
DS-01	Open system without dataset	UI opened for the first time	Open app without selecting dataset folder	System displays no folder selected and prevents progress to YOLO stage.
DS-02	Select valid dataset folder	A valid dataset folder exists	Select folder containing valid trajectories	Selected path is displayed and trajectory folders are detected.
DS-03	Select invalid dataset folder	Folder without valid trajectories exists	Select folder missing required structure	System displays clear validation error and prevents progress.
DS-04	Select parent dataset folder	Parent folder contains actual dataset subfolder	Select parent folder	System detects relevant dataset folder or explains required structure.
DS-05	Replace selected dataset	A dataset was already selected	Select a different dataset folder	Path, statistics, and dependent stage statuses update accordingly.
DS-06	Detect existing filtered CSV	Selected dataset contains a filtered CSV	Select the dataset folder	System detects CSV, displays status, and allows reuse if valid.
DS-07	Reject invalid CSV	CSV exists but required columns are missing	Attempt to reuse the CSV	System rejects invalid CSV and requires rebuilding through YOLO filtering.

10.2 YOLO Filtering Tests

Test ID	Feature	Preconditions	Test Steps	Expected Result
YOLO-01	Enter YOLO stage without existing CSV	Valid dataset selected and no filtered CSV exists	Navigate to YOLO stage	System displays NOT_STARTED status, dataset path, and start filtering option.
YOLO-02	Detect existing CSV	Valid filtered CSV already exists	Navigate to YOLO stage	System displays EXISTING or READY status without misleading new-run progress.
YOLO-03	Reuse existing CSV	Valid filtered CSV exists	Choose to use existing CSV	YOLO output is marked ready and OpenCLIP stage becomes available.
YOLO-04	Rebuild CSV using YOLO	Dataset selected	Start rebuild using YOLO	Scan starts from 0%, progress is shown, and next stage is blocked while running.
YOLO-05	YOLO progress update	Filtering is running	Wait during scan	Scanned, approved, rejected, skipped, and progress values update.
YOLO-06	Safe stop during YOLO scan	Filtering is running	Click Stop safely	System displays STOPPING and then STOPPED after safe termination.
YOLO-07	Resume YOLO scan	Scan stopped and partial results exist	Click Resume scan	Scan continues from saved partial CSV and does not restart from beginning.
YOLO-08	Restart YOLO scan	Stopped or partial scan exists	Click Restart from beginning	Partial CSV is replaced or removed and scan starts again from 0%.
YOLO-09	Detect partial YOLO state after reopening	YOLO stopped in middle	Close browser and reopen UI	System detects PARTIAL status and shows saved record count plus Resume/Restart.
YOLO-10	Return from OpenCLIP to YOLO	YOLO output ready and user moved forward	Navigate back to YOLO	YOLO stage displays READY status and existing summary without misleading logs.
YOLO-11	YOLO weights path display	YOLO configuration available	Open YOLO configuration area	Weights path is displayed read-only and cannot be edited accidentally.
YOLO-12	CSV last modified time	Filtered CSV exists	Open YOLO stage	System displays last modified time or relevant file metadata.

10.3 OpenCLIP Embedding Tests

Test ID	Feature	Preconditions	Test Steps	Expected Result
CLIP-01	Enter OpenCLIP stage without cache	YOLO output ready and no embeddings exist	Navigate to OpenCLIP stage	System displays missing embeddings and allows starting extraction.
CLIP-02	Detect complete embedding cache	Embeddings exist for all valid trajectories	Navigate to OpenCLIP stage	System displays EXISTING or READY status without misleading new-run progress.
CLIP-03	Reuse existing cache	Complete embedding cache exists	Choose existing cache	OpenCLIP stage is marked ready and Model

Test ID	Feature	Preconditions	Test Steps	Expected Result
				stage becomes available.
CLIP-04	Recompute embeddings	Complete or partial cache exists	Click Recompute all	New extraction starts from 0% and progress reflects the new run.
CLIP-05	OpenCLIP progress update	Extraction is running	Wait during extraction	Progress, processed trajectories, remaining trajectories, and status update.
CLIP-06	Safe stop during extraction	Extraction is running	Click Stop safely	System displays STOPPING and then STOPPED after safe termination.
CLIP-07	Resume extraction	Extraction stopped and partial embeddings exist	Click Resume extraction	System continues from existing embedding files and does not jump to 100% unless complete.
CLIP-08	Detect partial cache after reopening	Extraction stopped in the middle	Close browser and reopen UI	System detects PARTIAL cache and shows resume/recompute options.
CLIP-09	Return from Model to OpenCLIP	Cache accepted and user moved to Model	Navigate back to OpenCLIP	System displays READY status with correct summary and relevant logs only.
CLIP-10	Display cache timestamp	Embedding files exist	Open OpenCLIP stage	System displays last updated time or cache status information.
CLIP-11	OpenCLIP batch size configuration	OpenCLIP configuration visible	Open configuration area	Batch size is displayed as a controlled configuration value.
CLIP-12	Log stability during extraction	Extraction running and logs become long	Wait until log grows	Log scrolls inside its card without breaking layout.

10.4 Data Preparation Tests

Test ID	Feature	Preconditions	Test Steps	Expected Result
DATA-01	Load filtered dataset	Valid filtered CSV exists	Start dataset loading	System loads only valid records from the filtered CSV.
DATA-02	Status filter handling	CSV contains SUCCESS and/or FAILURE statuses	Load dataset with selected status filter	Dataset includes only samples matching selected status filter.
DATA-03	Load precomputed embeddings	OpenCLIP embeddings exist	Load dataset using precomputed embeddings	System loads embeddings from disk and does not recompute unnecessarily.
DATA-04	Missing embedding handling	At least one required embedding is missing	Load dataset	System reports missing embedding and handles it according to configuration.
DATA-05	Sequence window construction	Valid dataset and embeddings exist	Create windows with configured sequence length	System creates fixed-length windows and reports correct sample count.
DATA-06	Action target alignment	Valid state/action data exists	Build behavioral cloning samples	Each input window is paired with the correct target action.
DATA-07	Invalid sequence length	Invalid sequence length selected	Attempt to build windows	System prevents invalid configuration or displays a clear error.

10.5 Model, Training, and Checkpoint Tests

Test ID	Feature	Preconditions	Test Steps	Expected Result
TRAIN-01	Open Model stage after OpenCLIP ready	Dataset, YOLO, and OpenCLIP ready	Navigate to Model stage	System displays model configuration, hyperparameters, and checkpoint/training options.
TRAIN-02	Prevent training without required artifacts	CSV or embeddings missing	Attempt to start training	System prevents training and clearly reports missing artifacts.
TRAIN-03	Start new Mamba training	Dataset and embeddings available	Start training with selected hyperparameters	Training starts, logs are written, and progress/status is displayed.
TRAIN-04	Hyperparameter application	Hyperparameters configured	Start training	Training uses selected sequence length, d_model, batch size, learning rate, max epochs, and patience.
TRAIN-05	Training logs update	Training is running	Observe training log panel	Log displays epoch progress, validation information, and status messages.
TRAIN-06	Save checkpoint after training	Training completes successfully	Inspect checkpoints folder	Checkpoint file and configuration metadata are saved under correct run name.
TRAIN-07	Early stopping	Training runs with early stopping	Continue training until validation stops improving	Training stops according to patience and saves best available checkpoint.
TRAIN-08	Stop training safely	Training is running	Click stop if available	Process stops safely and UI reflects correct stopped state.

Test ID	Feature	Preconditions	Test Steps	Expected Result
TRAIN-09	Resume or continue training state	Training was stopped or existing run exists	Return to Model/Training stage	System displays correct run state and available actions without losing outputs.
CKPT-01	Detect available checkpoints	At least one checkpoint exists	Open checkpoint selection area	System lists available checkpoints.
CKPT-02	Display checkpoint metadata	Checkpoint has config metadata	Inspect checkpoint	System displays run name, model settings, and training configuration.
CKPT-03	Prevent evaluation without selected checkpoint	No checkpoint selected or file missing	Attempt evaluation	System prevents evaluation and displays a checkpoint error message.
CKPT-04	Use selected checkpoint for evaluation	Valid checkpoint exists	Select checkpoint and continue	Selected checkpoint is stored and used by evaluation stage.

10.6 Evaluation Tests

Test ID	Feature	Preconditions	Test Steps	Expected Result
EVAL-01	Open Evaluation stage with valid checkpoint	Valid checkpoint selected	Navigate to Evaluation	System displays evaluation configuration and allows running evaluation.
EVAL-02	Prevent evaluation without checkpoint	No valid checkpoint selected	Attempt evaluation	System prevents evaluation and displays clear message.
EVAL-03	Run evaluation	Valid dataset, embeddings, and checkpoint exist	Start evaluation	Evaluation process runs and displays progress/status updates.
EVAL-04	Compute evaluation metrics	Evaluation completes successfully	Inspect evaluation output	System computes and displays metrics such as MAE and sample count.
EVAL-05	Save prediction examples	Evaluation configured to save examples	Run evaluation	Prediction examples are saved and available in Results screen.
EVAL-06	Evaluation log generation	Evaluation running or completed	Open evaluation log	Log contains model loading, dataset loading, metrics, and completion messages.
EVAL-07	Handle failed evaluation	Missing files or incompatible configuration	Run under invalid conditions	System reports failure clearly and does not display misleading results.
EVAL-08	Re-run evaluation	Previous evaluation exists	Run evaluation again	System updates output and displays latest valid results.

10.7 Results Visualization Tests

Test ID	Feature	Preconditions	Test Steps	Expected Result
RES-01	Open Results stage after evaluation	Evaluation completed successfully	Navigate to Results	System displays evaluation results.
RES-02	Display metrics	Evaluation metrics exist	Open Results screen	Metrics such as MAE and sample count are displayed clearly.
RES-03	Display prediction charts	Prediction and ground truth values exist	Inspect charts area	Charts display predictions and ground truth without being cut off or misaligned.
RES-04	Enable chart interaction	Charts displayed	Use mouse zoom or pan	Chart supports expected interaction and remains visually stable.
RES-05	Display prediction example image	Examples include image/frame info	Open example viewer	Corresponding image or valid placeholder is displayed.
RES-06	Navigate prediction examples	Multiple examples exist	Click First/Previous/Next/Last	Displayed example changes correctly and counter/progress updates.
RES-07	Display action comparison table	Prediction and ground-truth vectors exist	Inspect table	Table displays dimensions, predictions, ground truth, and absolute error.
RES-08	Results layout consistency	Results include metrics, charts, images, controls, table, log	Inspect full screen	Elements remain within frames, aligned, readable, and not cut off.
RES-09	Display evaluation log	Evaluation log exists	Inspect log area	Log appears in correct section and does not overlap other content.

10.8 General Flow, State Persistence, and Error Handling Tests

Test ID	Feature	Preconditions	Test Steps	Expected Result
FLOW-01	Pipeline navigation	UI is open	Move between stages	Active stage is highlighted and only valid stages are accessible.
FLOW-02	Prevent progress during running process	A stage is running	Try to continue	System prevents progress until process completes or stops.

Test ID	Feature	Preconditions	Test Steps	Expected Result
FLOW-03	Back navigation after completed stage	At least one stage completed	Navigate back	Stage displays correct completed/ready status and retains artifacts.
FLOW-04	State restoration after reopening UI	Stage completed/stopped/partial/ready	Close and reopen UI	System restores correct state based on files, logs, and process status.
FLOW-05	New dataset resets dependent stages	Dataset already selected	Select new dataset	Dependent stages reset or update as needed.
FLOW-06	Logs separated by stage	Logs exist for multiple stages	Move between screens	Each screen displays only relevant logs.
FLOW-07	Missing artifact handling	Required artifact missing	Try dependent operation	System displays clear error and prevents invalid operation.
FLOW-08	UI layout consistency	UI contains buttons, charts, tables, images, logs	Inspect screens after pipeline run	Layout remains readable, aligned, and not cut off.
FLOW-09	Stop and resume consistency	Long process was stopped safely	Resume process	Process continues from saved partial state unless restart explicitly requested.
FLOW-10	Final end-to-end workflow	Valid dataset, models, and environment available	Run workflow from Dataset to Results	System completes pipeline and displays metrics, charts, examples, tables, and logs.

10.9 Data Consistency Validation Tests

Test ID	Feature	Preconditions	Test Steps	Expected Result
DATA-CONS-01	Validate state/action length relation	Trajectory has observations and actions	Compare counts	System confirms expected relation such as N observations and N-1 actions.
DATA-CONS-02	Validate action vector size	Policy output files available	Load action vectors	Each action vector has expected dimensionality for prediction.
DATA-CONS-03	Validate state vector size	Observation files available	Load state vectors	Each state vector has expected dimensionality and can be used as input.
DATA-CONS-04	Validate gripper-related component	State, desired state, and action vectors available	Compare last component over time	Last component behaves consistently with gripper open/close behavior.
DATA-CONS-05	Validate action-to-next-state alignment	State, desired state, and action sequences available	Compare action at t with target at following timestep	Selected alignment is consistent with behavioral cloning configuration.
DATA-CONS-06	Validate timestamp consistency	Timestamp data exists	Inspect ordering and gaps	Timestamps are ordered and do not contain invalid jumps.
DATA-CONS-07	Validate qpos/qvel consistency	Joint position and velocity data exist	Compare qpos changes with qvel	qvel behaves consistently with joint motion without invalid unexplained values.
DATA-CONS-08	Validate end-effector pose data	eef_transform data exists	Inspect transform matrix dimensions and values	End-effector transform has expected structure and can be used for validation when needed.
DATA-CONS-09	Validate filtered success trajectories	Filtered CSV exists	Count valid/SUCCESS trajectories	Filtered dataset contains expected usable trajectories for selected configuration.
DATA-CONS-10	Validate training window count	Dataset loading completed	Compare reported windows with configuration	Generated windows are consistent with valid trajectories, sequence length, and filtering.

11. Summary

This STP defines a structured testing plan for the RoboMamba system. It verifies both the machine-learning workflow and the engineering quality of the full pipeline: data preparation, YOLO-based curation, OpenCLIP embeddings, Mamba training/evaluation, checkpoint handling, Streamlit UI behavior, logging, state management, and final results visualization.